

# Warm-up: functions that we hand over

## Programming Paradigms

### Session 5 — the warm-up, before the quest

Part 1 goes with block 1 of the lecture, Part 2 with block 2.

In each part: questions on paper, in pairs, then exercises at the keyboard, with `warmup.ml`.

In the lecture a function was handed over three times: as one step of a traversal, as a computation not yet run, as the rest of a computation. Before the rope and the editor, we try the first two on things everyone has met: a list of numbers, a counter, a triangle of numbers, and the queue of lab 2. Nothing here is themed. The notation is OCaml's, and this is all we need:

---

|                                        |                                                                                     |
|----------------------------------------|-------------------------------------------------------------------------------------|
| <code>fun x -&gt; e</code>             | the function that takes <code>x</code> and gives <code>e</code> ; it has no name    |
| <code>let f x y = e</code>             | the same as <code>let f = fun x -&gt; fun y -&gt; e</code>                          |
| <code>f x y</code>                     | <code>f</code> applied to <code>x</code> , and the result applied to <code>y</code> |
| <code>fun () -&gt; e</code>            | a function that waits for nothing but its call: a thunk                             |
| <code>ref 0, !n, n := 1, incr n</code> | a cell that can change; read it; write it; add one                                  |
| <code>lazy e, Lazy.force l</code>      | a value computed at the first <b>force</b> , and kept                               |

---

The file `warmup/warmup.ml` holds the exercises at the keyboard, with holes marked `TODO` and several checks for each. The exercises are stated here; the file only says where to type. It needs no dune:

```
ocaml warmup/warmup.ml          # pass / FAIL / todo, one line per check
```

## Part 1 — functions, folds, iterators

### On paper

#### Question A (a function is a value)

Given the three definitions, write the value of each of the five expressions under them, and the types of `add`, of `add3` and of `twice`.

```
let twice f x = f (f x)
let add n = fun x -> x + n
let add3 = add 3

add3 10
twice add3 10
twice (add 1) 0
List.map (add 10) [1; 2; 3]
twice twice add3 10
```

Then, given the function `add_pair`, which takes its two numbers as one pair, write `add3` from it. Can we do it without writing a new function?

```
let add_pair (n, x) = x + n
```

#### Question B (what a function carries)

Given the function `counter` and the five definitions under it, write the values of `x`, `y` and `z`.

```
let counter () =
  let n = ref 0 in
  fun () -> incr n; !n
```

```

let a = counter ()
let b = counter ()
let x = a ()
let y = a ()
let z = b ()

```

Then, given the list of functions `fs`, write the value of the last expression.

```

let fs = List.map (fun i -> fun () -> i * 10) [1; 2; 3]

List.map (fun f -> f ()) fs

```

## At the keyboard

### Exercise 1 (functions that take and return functions)

Given `add` and `add_pair`, write `add3` from `add`, and `add3'` from `add_pair`. Then write `twice`, which applies a function two times, and `compose`, which applies `g` and then `f`.

```

add3      : int -> int
add3'     : int -> int
twice     : ('a -> 'a) -> 'a -> 'a
compose   : ('b -> 'c) -> ('a -> 'b) -> 'a -> 'c

```

### Exercise 2 (a closure with a state of its own)

Given `counter`, write `accumulator`: each call of `accumulator ()` makes a function with a total of its own, which starts at 0. That function adds its argument to the total and returns the total.

```

accumulator : unit -> int -> int

```

### Exercise 3 (three functions, written with a fold)

Given `fold`, write the sum of a list of numbers, the number of elements that satisfy a predicate, and the list in the opposite order. Each is one call of `fold` and has no recursion of its own.

```

let rec fold f acc l =
  match l with
  | [] -> acc
  | x :: xs -> fold f (f acc x) xs

sum      : int list -> int
count    : ('a -> bool) -> 'a list -> int
rev      : 'a list -> 'a list

```

### Exercise 4 (a fold that can stop)

Given `fold_until`, whose function answers `Continue` to go on and `Stop` to end the fold, write `find_first`, the first element that satisfies a predicate, and `sum_until_negative`, the sum of the numbers before the first negative one. Each is one call of `fold_until`.

```

type 'b go =
  | Continue of 'b
  | Stop of 'b

let rec fold_until f acc l =
  match l with
  | [] -> acc
  | x :: xs ->
    match f acc x with
    | Stop a -> a
    | Continue a -> fold_until f a xs

```

```
find_first      : ('a -> bool) -> 'a list -> 'a option
sum_until_negative : int list -> int
```

One check counts the calls of the predicate: on `[3; 12; 7; 40]`, looking for a number over 10, there are two.

### Exercise 5 (an iterator, as a closure)

Given two numbers `a` and `b`, write `range`, the iterator that gives `a`, then `a + 1`, up to `b - 1`, one number at each call, and `None` at every call after that. Then write `drain`, the list of the elements that an iterator still has to give.

```
range : int -> int -> unit -> int option
drain : (unit -> 'a option) -> 'a list
```

## Part 2 — thunks, sequences, lazy

### On paper

#### Question C (when is it printed?)

Given the program, write what it prints, character by character, in order.

```
let t = fun () -> print_string "A"; 1
let l = lazy (print_string "B"; 2)

let () = print_string "C"

let x = t () + t ()
let y = Lazy.force l + Lazy.force l

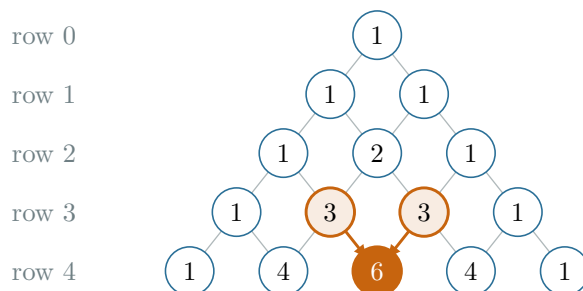
let () = Printf.printf " %d %d" x y
```

#### Question D (a triangle of numbers)

In Pascal's triangle every row begins and ends with 1, and every other number is the sum of the two numbers above it. We write  $T(n, k)$  for the number of row  $n$  at column  $k$ , where the rows and the columns are counted from 0. The triangle is defined by recursion on  $n$ :

$$\begin{aligned} T(n, 0) &= 1 \\ T(n, n) &= 1 \\ T(n, k) &= T(n-1, k-1) + T(n-1, k) \quad \text{for } 0 < k < n \end{aligned}$$

In the drawing,  $T(4, 2) = T(3, 1) + T(3, 2)$  is marked:  $6 = 3 + 3$ .



The same definition, read one row at a time: we write row  $n - 1$  twice, the first time with a 0 before it and the second time with a 0 after it, and we add the two column by column. The result is row  $n$ .

```

      0  1  3  3  1
+     1  3  3  1  0
=     1  4  6  4  1

```

Given the row 1 4 6 4 1, write the two rows that follow. Then say, for the first line of the addition and for the second one, which number of the definition stands at column  $k$ , and where the two 1 at the ends of a row come from. Last, write the sum of each of the rows 0 to 5. What are these sums?

## At the keyboard

The type of sequences is the lecture's, and `take` is given.

```

type 'a node =
| Nil
| Cons of 'a * 'a seq

and 'a seq = unit -> 'a node

```

### Exercise 6 (a row of the triangle, from the row above it)

Given two lists, write `zip_with`, which combines them element by element with a function and stops at the end of the shorter one. Then write `next_row`, by the rule of question D, as one call of `zip_with`.

```

zip_with : ('a -> 'b -> 'c) -> 'a list -> 'b list -> 'c list
next_row : int list -> int list

```

*From the definition to the code.* If `row` is row  $n - 1$ , the list `0 :: row` has  $T(n - 1, k - 1)$  at position  $k$ , and the list `row @ [0]` has  $T(n - 1, k)$  there. The three equations of the definition become one addition of two lists: `next_row` has no `if` and no `case` for the ends of the row.

### Exercise 7 (a sequence without end)

Given a function `f` and a value `x`, write `iterate`, the sequence `x, f x, f (f x)`, and so on, without end. The line that counts the cells built is given; only the cell is to write.

```

iterate : ('a -> 'a) -> 'a -> 'a seq

```

One check makes the sequence and reads the counter before anything is asked for: it is 0.

*From the definition to the code.* A definition by recursion on  $n$  has a first value and a step from  $n - 1$  to  $n$ . `iterate` receives exactly these two: `x` is the first value and `f` is the step.

### Exercise 8 (the triangle, as the sequence of its rows)

Given `iterate` and `next_row`, write `pascal`, the sequence of all the rows of the triangle, and `row`, which returns row `n`, the top row being row 0.

```

pascal : int list seq
row     : int -> int list

```

*From the definition to the code.* Row 0 is the list with the single number 1, and `next_row` is the step from row  $n - 1$  to row  $n$ . The definition says how to compute any row; it does not say how many. With a sequence we do not have to say it either.

One check counts the rows built by `row 6`: there are seven, and no more. When `pascal` is written, the file draws the triangle on the screen, two times: with its numbers, and with a star for every odd number.

### Exercise 9 (the diagonals of the triangle)

Given the rows of the triangle, write the one line that is missing in `diagonal`. Diagonal `n` starts in row `n` at column 0 and goes up to the right: row `n - 1` at column 1, row `n - 2` at column 2, as long as the column exists in the row. `diagonal n` is the sum of these numbers:

$$T(n, 0) + T(n - 1, 1) + T(n - 2, 2) + \dots$$

In the code, `k` is the column and `acc` the sum so far;  $T(n - k, k)$  is element `k` of element `n - k` of the list `rows`.

```
diagonal : int -> int
```

Then write the first ten sums on paper. Where have we seen these numbers?

### Exercise 10 (a row computed once)

Given `row`, write `lazy_rows`: the list of `n` lazy values, where number `i` holds row `i`, not yet computed.

```
lazy_rows : int -> int list Lazy.t list
```

The checks make ten lazy rows and read the counter: no row is built. Then they read row 5 two times: six rows are built at the first reading, and none at the second.

*From the definition to the code.* The definition can also be written as it stands, as a function `t n k` with two recursive calls. That function computes the same numbers again and again, as the list of Fibonacci numbers did with thunks:  $T(3, 1)$  is needed by  $T(4, 1)$  and by  $T(4, 2)$ . A row that is kept is computed once for all the numbers under it.

## Homework — lab 2's queue, with its reversal kept

This part is not for the lab session. It goes with the end of block 2, and it answers the question on which lab 2 ended.

### On paper

#### Question E (a queue used twice)

Given lab 2's queue, a front and a rear, where `pop` reverses the rear when the front is empty, and the queue `q`, write the values of `a` and of `b`. How many times is the list `[3; 2; 1]` reversed?

```
let pop q =
  match q with
  | x :: front, rear -> Some (x, (front, rear))
  | [], rear ->
    match reverse rear with
    | [] -> None
    | x :: front -> Some (x, (front, []))

let q = ([], [3; 2; 1])

let a = pop q
let b = pop q
```

Then, given that the reversal is kept in the queue as a lazy value, which `a` and `b` both read, write how many times it is run.

### At the keyboard

#### Exercise 11 (lab 2's queue, with its reversal kept)

Given the type of the second queue, where the front is a lazy value and the two lengths are kept, write the two expressions that are missing. In `balance`: the new front, which is the old front followed by the rear reversed, and which must not be computed now. In `lpop`: the queue without its first element.

```
type 'a lqueue = {
  front : 'a list Lazy.t;
  flen : int;
```

```
    rear : 'a list;  
    rlen : int;  
}  
  
balance : 'a lqueue -> 'a lqueue  
lpush    : 'a -> 'a lqueue -> 'a lqueue  
lpop     : 'a lqueue -> ('a * 'a lqueue) option
```

The file is `warmup/queue.ml`, and it is run as the other one. Its first check is question E, on lab 2's queue as it was. Its last check pops the same lazy queue two times: the second time, no reversal is run.